

Information Retrieval Notes

Academic Year 2026/2027

Contents

1	Introduction and Architecture of Information Retrieval	1
1.1	Definition and Motivations	1
1.1.1	The Generative Shift: From Web Search to Agentic Search	2
1.1.2	Information Retrieval vs. Data Retrieval	3
1.2	The Two Fundamental Goals: Effectiveness and Efficiency	5
1.2.1	Efficiency and Underlying Hardware Architecture	5
1.3	Information Retrieval System Architecture	6
1.3.1	Text Processing Pipeline	7
1.4	Language Properties and Retrieval Models	7
1.5	The Inverted Index	8
1.5.1	Query Processing and Merge Algorithms	9
1.6	Index Compression Techniques	11
1.6.1	Binary Interpolative Coding	11
1.6.2	Dictionary-Based Compression: LZ77	12
1.7	Learning-to-Rank (LTR) and the Non-Differentiability Challenge	13
1.7.1	Formulation of the Ranking Problem	13
1.7.2	The Theoretical Challenge: Non-Differentiability of Ranking	14
1.7.3	LTR Model Families and Surrogate Gradients	15
1.8	Evolution Toward Neural Information Retrieval	16
1.8.1	Three Embedding Families	17
1.9	k -Approximate Nearest Neighbor Search (k -ANN) and Scalability	18
1.9.1	Forward Index and the TusKANNy Framework	18
2	Evaluation of Information Retrieval Systems	20
2.1	Search Engine Architecture	20
2.2	Evaluating Search Engine Quality	22
2.3	Measuring User Happiness	23
2.4	The Cranfield Paradigm and Offline Test Collections	25
2.4.1	Combinatorial Scale Bottlenecks	25

2.4.2	Origins of Test Queries	26
2.4.3	Crowdsourcing: Opportunities and Pitfalls	26
2.4.4	The TREC Infrastructure	27
2.4.5	The Pooling Technique	28
2.5	Information Need vs. Query	29
2.6	Unranked Retrieval Measures	29
2.6.1	Precision, Recall, and the Precision-Recall Trade-off	29
2.6.2	The Harmonic Mean: F-Measure (F_1 -Score)	30
2.6.3	The Weighted F_β -Measure	31
2.7	Rank-Based Measures (Binary Relevance)	31
2.7.1	Precision@k ($P@k$) and Recall@k ($R@k$)	32
2.7.2	Average Precision (AP)	33
2.7.3	Mean Average Precision (MAP)	35
2.7.4	Truncated Ranking Measures: AP@k and MAP@k	36
2.7.5	Mean Reciprocal Rank (MRR)	36
2.8	Beyond Binary Relevance: Graded Relevance and NDCG	37
2.8.1	Cumulative Gain (CG) and Discounted Cumulative Gain (DCG)	38
2.8.2	Normalized Discounted Cumulative Gain	39
2.8.3	Complete Numerical Walkthrough	39
2.8.4	Comparing Alternative Ranking Functions	39

Chapter 1

Introduction and Architecture of Information Retrieval

1.1 Definition and Motivations

Information Retrieval (IR) is the scientific and engineering discipline concerned with representing, storing, organizing, and accessing large collections of predominantly unstructured or semi-structured data to satisfy an **information need** expressed by a user [1, 2].

Definition 1.1.1 (Information Retrieval (Manning et al.)). Information Retrieval is finding material of an unstructured nature (usually text) that satisfies an information need from within large collections stored on computers.

In the modern digital landscape, Information Retrieval extends far beyond general-purpose Web search engines (such as Google or Bing) to serve as the foundational infrastructure for:

- **E-commerce and marketplaces:** product retrieval driven by textual descriptions, structured specifications, and customer reviews (e.g., Amazon, Alibaba, eBay).
- **Enterprise and Desktop Search:** indexing and semantic searching across enterprise repositories (documents, emails, source code bases).
- **RAG (*Retrieval-Augmented Generation*) Systems:** grounding Large Language Models (LLMs) with external, authoritative context documents to eliminate hallucinations and supply real-time fresh knowledge.

- **Autonomous agents and recommender engines:** content-based and collaborative filtering for personalized content discovery and media streaming (e.g., Netflix, Spotify, Meta, Pinterest).

1.1.1 The Generative Shift: From Web Search to Agentic Search

The rise of conversational Large Language Models (LLMs) has not diminished the relevance of Information Retrieval; rather, it has placed IR at the center of modern AI systems. An interview with an LLM reveals that conversational AI fundamentally depends on core IR competencies:

1. **Semantic Query Understanding:** advanced natural language processing (NLP) to parse and disambiguate user intent.
2. **Matching and Relevance Scoring:** identifying and prioritizing the most pertinent information across massive corpora.
3. **Contextual Retrieval for Fresh Information:** pure generative models are bound by their training cutoff and cannot answer questions requiring recent or private knowledge without expensive retraining.

To bridge this gap, modern architectures rely on **RAG (*Retrieval-Augmented Generation*)**: the system extracts queries from user input, retrieves authoritative passages from an external corpus or the Web, and injects them into the LLM’s prompt context to synthesize an accurate, cited response without modifying model parameters.

Agentic Search and the Adaptive Feedback Loop

While conventional RAG operates as a single-step pipeline (query → retrieve → generate), complex inquiries necessitate **Agentic Search**:

Definition 1.1.2 (Agentic Search). **Agentic Search** is an autonomous, multi-turn retrieval process in which an AI agent dynamically decides subsequent search actions based on intermediate findings. The agent inspects retrieved sources, identifies missing knowledge or discrepancies (*information gaps*), formulates targeted follow-up queries, and iterates until sufficient evidence is assembled to synthesize the final answer.

For example, when prompted with: *"Why did Company X's profits drop last quarter despite record revenue?"*, an agentic system retrieves the financial

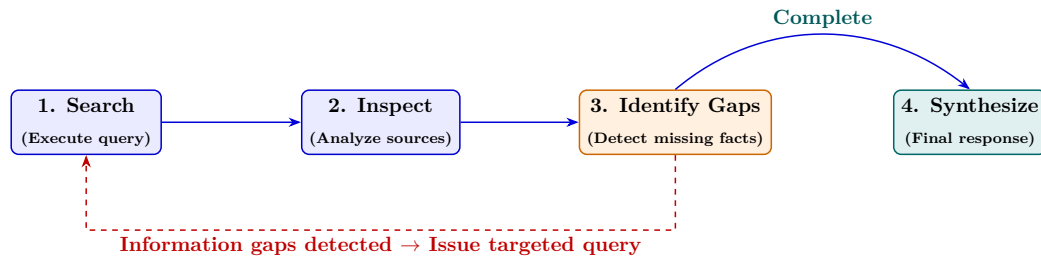


Figure 1.1: Adaptive feedback loop characterizing Agentic Search.

report, notices the divergence between revenue and operating margins, identifies the need to investigate rising raw material costs or restructuring expenses, executes a refined query, and consolidates the evidence into an analytical answer with citations.

1.1.2 Information Retrieval vs. Data Retrieval

A fundamental conceptual distinction lies in contrasting traditional relational database querying (*Data Retrieval*) with Information Retrieval:

Relevance: The Core of IR

While database systems evaluate binary truth against relational predicates, Information Retrieval focuses on estimating the degree of **relevance** of each document with respect to the user's underlying cognitive information need. Relevance is inherently subjective, context-dependent, and time-varying.

Dimension	Data Retrieval (SQL Databases)	Information Retrieval
Data Structure	Highly structured data with strict relational schemas (tables, types, constraints)	Unstructured or semi-structured data (free text, HTML, PDF, metadata)
Query Language	Formal deterministic queries (e.g., SQL) with exact relational algebra semantics	Natural language keywords or phrases (often ambiguous, short, and incomplete)
Matching Criterion	Exact Boolean matching: a record either strictly satisfies or violates the predicate	Partial and probabilistic matching: documents exhibit varying degrees of relevance
Result Ordering	No intrinsic ordering (unless explicitly specified via <code>ORDER BY</code>)	Central ranking mechanism driven by relevance estimation (<i>Relevance Ranking</i>)
Error Tolerance	Zero tolerance: syntax errors or unexpected values produce errors or empty sets	High tolerance: resilience to typos, synonyms, polysemy, and morphological variants

Table 1.1: Analytical comparison between traditional Data Retrieval and Information Retrieval.

1.2 The Two Fundamental Goals: Effectiveness and Efficiency

A large-scale information retrieval system must simultaneously optimize two competing objectives:

1. **Effectiveness:** Quantifies the intrinsic quality of the returned results, assessing how well the ranked list satisfies the user’s search intent (thoroughly addressed in Chapter 2).
2. **Efficiency:** Quantifies computational cost, throughput (queries processed per second), and latency (demanded within 10–50 milliseconds per query across billions of documents).

1.2.1 Efficiency and Underlying Hardware Architecture

The efficiency of a modern search engine depends not merely on algorithmic complexity, but on hardware execution dynamics:

- **Pipelining and Superscalar Processors:** Modern CPUs segment instruction execution into sequential stages (Instruction Fetch, Decode, Execute, Memory Access, Write Back), issuing multiple instructions per clock cycle.
- **Pipeline Hazards:**
 - *Structural Hazards:* Resource contention when multiple instructions compete for the same execution unit or memory bus.
 - *Data Hazards:* Logical data dependencies between consecutive instructions (RAW: Read-After-Write), forcing pipeline stalls (*bubbles*) when values are unavailable.
 - *Control Hazards:* Branch mispredictions. During posting list intersection loops, an incorrect branch prediction flushes the pipeline, incurring a penalty of 15–20 clock cycles per misprediction.
- **Memory Hierarchy:** CPU registers operate at sub-nanosecond speeds, L1/L2/L3 caches respond in a few clock cycles, whereas main memory (RAM) requires 50–100 ns and secondary storage (NVMe/SSD) demands tens of microseconds. Posting list layouts prioritize sequential, contiguous memory layouts to maximize cache-line reuse and trigger hardware prefetching.

- **SIMD (*Single Instruction Multiple Data*) Vectorization:** Utilizing 128-bit, 256-bit, or 512-bit vector registers (AVX-2, AVX-512, ARM NEON) enables simultaneous comparisons of 8 or 16 document identifiers in a single machine cycle, dramatically accelerating posting intersections.
- **Parallel Computing and Distributed Sharding:** Indices exceeding the capacity of a single machine are partitioned (*sharded*), typically through document-level sharding, allowing compute clusters to process queries concurrently with centralized result aggregation.

1.3 Information Retrieval System Architecture

As depicted in Figure 1.2, an IR system enforces a strict decoupling between the **Offline Pipeline** (indexing and model training) and the **Online Pipeline** (low-latency query serving).

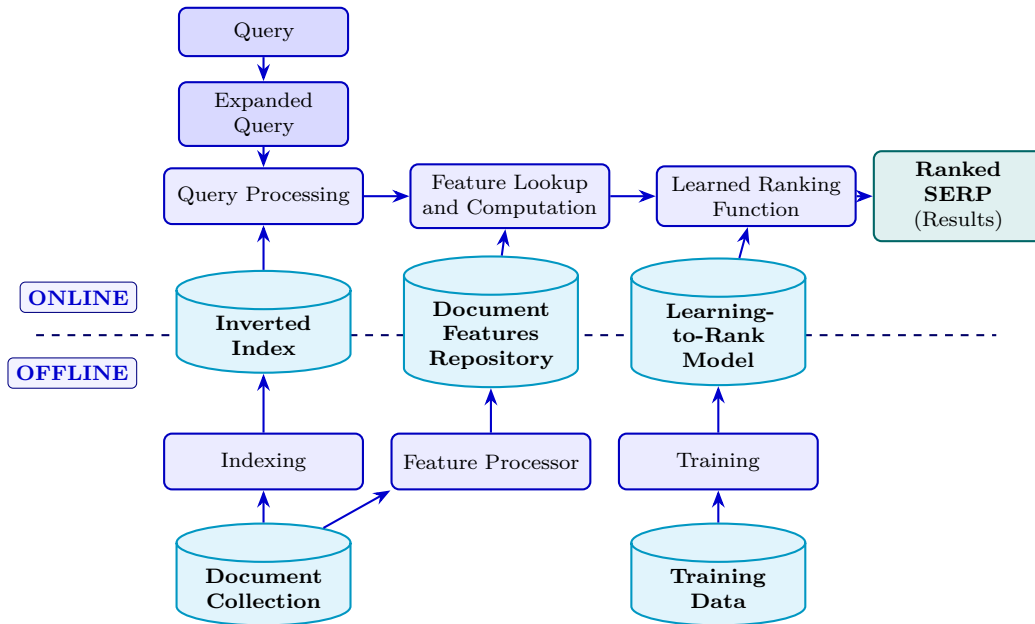


Figure 1.2: Grid architecture of a modern search engine (Online vs. Offline pipelines).

1.3.1 Text Processing Pipeline

Raw textual content undergoes four sequential preparation stages:

1. **Tokenization:** Decomposing character streams into lexical tokens, handling punctuation, special characters, and hyphenated terms.
2. **Normalization and Case Folding:** Converting tokens to lowercase and stripping accents/diacritics to ensure uniformity across queries and documents.
3. **Stopword Filtering:** Removing high-frequency functional words with minimal discriminative value (e.g., "the", "is", "at", "which").
4. **Stemming and Lemmatization:**
 - *Stemming:* Algorithmic stripping of suffixes using heuristic rules (such as Porter's algorithm [1]) to extract the root form (*stem*). Example: "retrieving", "retrieval" → "retriev".
 - *Lemmatization:* Full grammatical analysis utilizing vocabulary dictionaries to map inflected forms to their formal base lemma. Example: "am", "were" → "be".

1.4 Language Properties and Retrieval Models

Natural language word distributions adhere to statistical empirical laws that shape index designs and ranking algorithms:

- **Zipf's Law:** The frequency $f(r)$ of a term is inversely proportional to its frequency rank r :

$$f(r) \propto \frac{1}{r} \quad (1.1)$$

A handful of words dominate overall occurrences, while the vast majority of words appear extremely rarely (*long tail*).

- **Heaps' Law:** The vocabulary size $|V|$ grows as a power law with respect to collection size in terms of total word tokens N :

$$|V| = k \cdot N^\beta \quad (\text{with } 0.4 \leq \beta \leq 0.6) \quad (1.2)$$

The vocabulary never fully saturates as more documents are gathered.

Core retrieval models include:

1. **Boolean Model:** Deterministic matching against logical operators ($\wedge, \vee, \neg$), lacking an intrinsic relevance score.
2. **Vector Space Model (TF-IDF):** Documents and queries are modeled as vectors across vocabulary space $|V|$, weighted by term frequency (TF) and inverse document frequency (IDF), scored via cosine similarity.
3. **Probabilistic Model (BM25):** The de facto industry standard for lexical retrieval based on the 2-Poisson probabilistic model [2]. It formalizes asymptotic term frequency saturation and penalizes excessively long documents to mitigate verbosity bias:

$$\text{score}_{\text{BM25}}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{tf(t, d) \cdot (k_1 + 1)}{tf(t, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)} \quad (1.3)$$

where:

- $\text{IDF}(t) = \ln \left(\frac{N - df_t + 0.5}{df_t + 0.5} + 1 \right)$ quantifies the informative specificity of term t across the entire collection of N documents.
- $tf(t, d)$ is the raw term frequency of t within document d .
- $k_1 \geq 0$ (term frequency saturation parameter, typically calibrated between 1.2 and 2.0): governs how rapidly repeated occurrences of the same term flatten toward an asymptote.
- $b \in [0, 1]$ (document length normalization penalty, typically $b \approx 0.75$): regulates the penalty applied to document length $|d|$ relative to the average collection document length avgdl . At $b = 1$, the penalty for lengthy documents is maximal; at $b = 0$, document length is entirely ignored.

1.5 The Inverted Index

The **Inverted Index** reverses document storage directionality, establishing a mapping $t \mapsto \mathcal{P}_t$, where $\mathcal{P}_t$ is the ordered list of documents containing term t [1].

Definition 1.5.1 (Inverted Index). An Inverted Index consists of two complementary components:

1. **Dictionary (Vocabulary):** Stores all unique terms appearing across the collection, maintained via fast lookup structures (B-Trees or Hash Tables), along with document frequency df_t and a pointer to the start of its posting list.
2. **Posting List:** For each term t , an ordered array of document identifiers sorted strictly in ascending order ($\text{DocID}_1 < \text{DocID}_2 < \dots$). Each posting may also record auxiliary metadata such as local term frequencies (tf) and intra-document term position offsets.

Example 1.5.1 (Reference Toy Collection). Consider a toy educational collection of three documents:

- d_1 : "An inverted index is an index storing a mapping from terms to their occurrences in a set of documents."
- d_2 : "The inverted index data structure is a central component of a typical search engine indexing algorithm."
- d_3 : "The goal is to optimize the speed of the query: find the documents where a set of words occur."

Figure 1.3 illustrates the Inverted Index constructed for the key terms extracted from this collection.

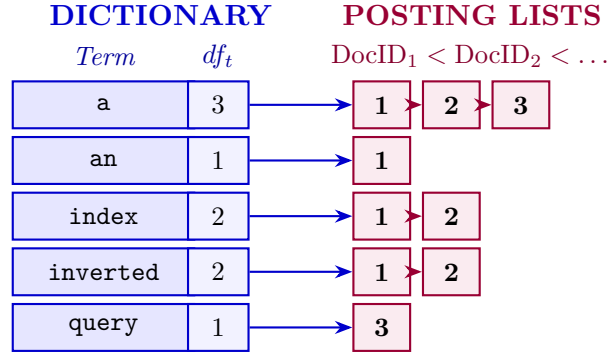


Figure 1.3: Inverted Index structure for the reference toy collection (d_1, d_2, d_3): Term dictionary with document frequency df_t and linked posting lists.

1.5.1 Query Processing and Merge Algorithms

The strictly ascending order of DocIDs enables Boolean and phrase queries to be evaluated via coordinated traversals of posting lists:

- **Conjunctive Query (AND):** Identifies documents containing all specified terms simultaneously. For example, evaluating $q = \text{"inverted" AND "index"}$:

$$\mathcal{P}_{\text{inverted}} \cap \mathcal{P}_{\text{index}} = [1, 2] \cap [1, 2] = [1, 2] \implies \{d_1, d_2\}$$

Conversely, for $q = \text{"query" AND "inverted" AND "index"}$:

$$(\mathcal{P}_{\text{inverted}} \cap \mathcal{P}_{\text{index}}) \cap \mathcal{P}_{\text{query}} = [1, 2] \cap [3] = \emptyset$$

The two-pointer intersection merge (Algorithm 1) provides linear complexity $\mathcal{O}(|P_1| + |P_2|)$. When evaluating queries with $k > 2$ terms, best practice dictates processing posting lists in ascending order of document frequency df_t , starting with the shortest list to minimize intermediate candidate sets.

- **Disjunctive Query (OR):** Identifies documents containing at least one of the query terms. For $q = \text{"query" OR "inverted"}$:

$$\mathcal{P}_{\text{query}} \cup \mathcal{P}_{\text{inverted}} = [3] \cup [1, 2] = [1, 2, 3] \implies \{d_1, d_2, d_3\}$$

- **Phrase Search:** To match exact sequences of words (e.g., "inverted index"), posting lists store positional offsets for each occurrence within a document. The evaluation algorithm verifies that:

$$\text{pos}(\text{index}) - \text{pos}(\text{inverted}) = 1$$

filtering out documents where terms appear in reversed order or at non-adjacent positions.

Algorithm 1: Merge Intersection for Conjunctive Queries ($P_1 \wedge P_2$)

Input: Two sorted posting lists P_1 and P_2

Output: Sorted list R containing DocIDs present in both lists

```

1  $R \leftarrow \emptyset$ 
2  $i \leftarrow 1; j \leftarrow 1$ 
3 while  $i \leq |P_1|$  and  $j \leq |P_2|$  do
4   if  $P_1[i] == P_2[j]$  then
5      $R.append(P_1[i])$ 
6      $i \leftarrow i + 1$ 
7      $j \leftarrow j + 1$ 
8   else if  $P_1[i] < P_2[j]$  then
9      $i \leftarrow i + 1$ 
10  else
11     $j \leftarrow j + 1$ 
12 return  $R$ 

```

From Theory to Practice: Google Advanced Search

Theoretical Boolean and phrase retrieval constructs directly map to commercial search engine operators and advanced query interfaces (Table 1.2).

IR Logical Operator	Google Advanced Search Field	Operational Syntax and Example
Conjunction (AND)	<i>all these words</i>	Juxtaposed keywords: tri-colour rat terrier
Phrase Search	<i>this exact word or phrase</i>	Double quotes: "rat terrier"
Disjunction (OR)	<i>any of these words</i>	Uppercase operator: miniature OR standard
Negation (NOT)	<i>none of these words</i>	Minus prefix without spaces: -rodent -"Jack Russell"
Numeric Range	<i>numbers ranging from ... to</i>	Two-dot operator: 10..35 kg, £300..£500

Table 1.2: Direct mapping between theoretical Query Processing constructs and commercial Advanced Search interfaces.

1.6 Index Compression Techniques

Compressing inverted indices serves two goals: minimizing storage footprint and converting slow I/O bus transfer bottlenecks into fast CPU decompression operations.

1.6.1 Binary Interpolative Coding

Binary Interpolative Coding (introduced by Moffat and Stuiver [3]) is a specialized compression algorithm tailored for strictly monotonically increasing integer sequences.

Definition 1.6.1 (Interpolative Coding Principle). Given a sorted subsequence of n integers $L = [x_1, x_2, \dots, x_n]$ bounded in the interval $[l, h]$ (such that $l \leq x_1 < x_2 < \dots < x_n \leq h$), the algorithm selects the median element $m = \lfloor (n+1)/2 \rfloor$. Because there must be at least $m-1$ distinct integers preceding x_m and at least $n-m$ distinct integers following it, x_m is restricted to the interval:

$$l + m - 1 \leq x_m \leq h - n + m \quad (1.4)$$

The number of valid candidate values for x_m is:

$$R = (h - n + m) - (l + m - 1) + 1 = h - l - n + 1 \quad (1.5)$$

The shifted value $(x_m - (l + m - 1))$ is encoded using exactly $\lceil \log_2 R \rceil$ bits. The procedure then recurses on the left sub-array over $[l, x_m - 1]$ and on the right sub-array over $[x_m + 1, h]$.

Example 1.6.1 (Recursive Decomposition Tree). Consider the sorted posting list of $n = 12$ identifiers bounded in the interval $[0, 62]$:

$$L = [3, 4, 7, 13, 14, 15, 21, 25, 36, 38, 54, 62] \quad (n = 12, m = 6, l = 0, h = 62)$$

The recursive decomposition tree (Figure 1.4) partitions the sequence hierarchically:

- The median element $x_6 = 15$ splits the sequence into left sublist $[3, 4, 7, 13, 14]$ constrained by $(n = 5, m = 3, l = 0, h = 14)$ and right sublist $[21, 25, 36, 38, 54]$ constrained by $(n = 5, m = 3, l = 16, h = 62)$.
- At the next lower level, the first sublist splits into $[3, 4]$ with $(n = 2, m = 1, l = 0, h = 6)$ and $[13, 14]$ with $(n = 2, m = 1, l = 8, h = 14)$.
- The leaf nodes encode individual elements: for instance, for $[14]$ with parameters $(n = 1, m = 1, l = 14, h = 14)$, the candidate range of valid values collapses to:

$$R = 14 - 14 - 1 + 1 = 1 \implies \lceil \log_2 1 \rceil = 0 \text{ bits!}$$

When document identifiers form dense clusters, the candidate range R collapses to 1, reducing the storage footprint to zero bits for many clustered postings.

1.6.2 Dictionary-Based Compression: LZ77

For general text or token streams, the **Lempel-Ziv '77** (LZ77) algorithm maintains a sliding window comprising a Search Buffer and a Lookahead Buffer, emitting triples formatted as (offset, length, next_char).

Consider the phrase segmentation of the sample input sequence:

$$T = \underbrace{a}_{(0, 'a')} \underbrace{b}_{(0, 'b')} \underbrace{a \ b \ a}_{(2, 3)} \underbrace{a \ a \ a}_{(1, 3)} \underbrace{b \ a \ b \ a \ a \ a \ a \ b}_{(7, 8)}$$

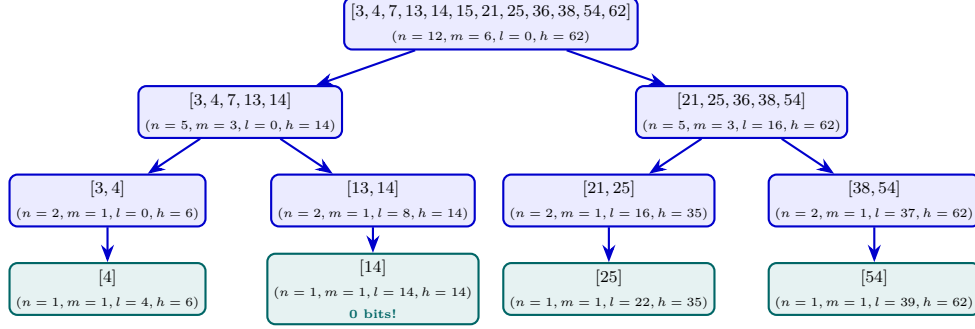


Figure 1.4: Dichotomous decomposition tree for Binary Interpolative Coding.

The resulting sequence of emitted pointers is:

$$(0, 'a'), (0, 'b'), (2, 3), (1, 3), (7, 8)$$

In this manner, long lexical repetitions are replaced with compact backward references within the sliding window. This algorithmic family forms the core of widely used production compressors such as `gzip`, `7-Zip`, and `LZ4` for high-throughput compressed storage.

1.7 Learning-to-Rank (LTR) and the Non-Differentiability Challenge

Within the architecture of a modern search engine (cf. Figure 1.2), the candidate documents retrieved by the Inverted Index are passed to a machine learning re-ranking stage known as **Learning-to-Rank** (LTR) [4].

1.7.1 Formulation of the Ranking Problem

In the supervised LTR setting, the training collection consists of a set of queries, each paired with a list of candidate documents evaluated by human assessors or derived from implicit click logs:

$$\mathcal{T} = \{(q, \langle (d_1, y_1), (d_2, y_2), \dots, (d_m, y_m) \rangle)\} \quad (1.6)$$

where $y_i \in \{0, 1, 2, 3, 4\}$ denotes the graded relevance judgment on a multi-level ordinal scale:

- $y = 0$: Irrelevant;
- $y = 1$: Marginally relevant;

- $y = 2$: Relevant;
- $y = 3$: Highly relevant;
- $y = 4$: Perfectly relevant.

Given a feature vector extracted for each query-document pair $\vec{x}_{q,d} = \Phi(q, d)$, a machine learning model computes a predicted relevance score $s_i = f_\theta(\vec{x}_{q,d_i})$. The final ranked list presented to the user is generated by sorting the documents in descending order of predicted score:

$$\pi = \text{sort}_\downarrow(\{(d_i, s_i)\}_{i=1}^m) \quad (1.7)$$

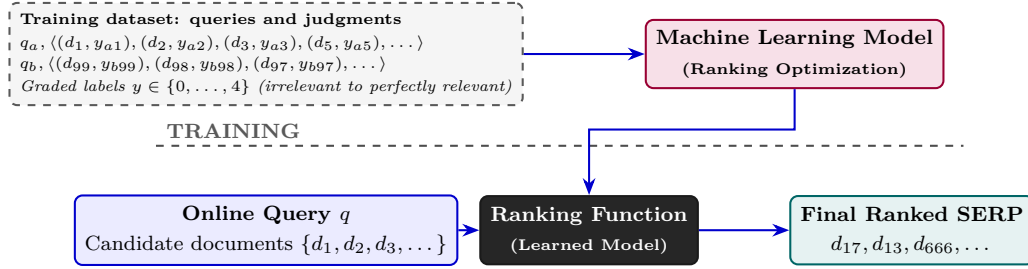


Figure 1.5: Conceptual workflow of Learning-to-Rank (LTR): from training with graded labels to online SERP ordering.

1.7.2 The Theoretical Challenge: Non-Differentiability of Ranking

In standard regression problems, an algorithm minimizes mean squared error $\sum_i (f_\theta(x_i) - y_i)^2$. In Information Retrieval, however, a foundational principle applies:

The Core Principle of LTR

The goal is to learn the ranking, not the label: the primary objective is not accurately predicting an isolated numerical label y , but optimizing the relative ordering of documents in the list presented to the user, maximizing evaluation metrics such as **NDCG** (*Normalized Discounted Cumulative Gain*) or **MAP** (*Mean Average Precision*).

This formulation introduces a fundamental mathematical obstacle:

- Ranking metrics $\mathcal{M}(\pi)$ depend directly on the **discrete ranks** (positions $1, 2, \dots, k$) assigned to each document by the sorting permutation.
- The sorting operation (sort) is a piecewise constant, discontinuous step function.
- As model parameters θ vary smoothly, document positions remain invariant (derivative is exactly zero) until two scores cross, creating a step discontinuity (derivative is undefined):

$$\frac{\partial \mathcal{M}_{\text{NDCG}}(\pi_\theta)}{\partial \theta} = 0 \quad \text{almost everywhere (a.e.)} \quad (1.8)$$

- Consequently, **standard stochastic gradient descent (SGD) and backpropagation cannot be applied directly** to optimize official ranking metrics!

1.7.3 LTR Model Families and Surrogate Gradients

To circumvent the non-differentiability barrier, the literature has established three primary methodologies:

1. **Pointwise Approach:** Treats each document as an independent regression or ordinal classification instance. While standard SGD applies directly, this method fails to model the inherently relative, competitive nature of ranked lists.
2. **Pairwise Approach (e.g., RankNet):** Considers pairs of documents (d_i, d_j) for the same query with differing relevance $(y_i > y_j)$. The objective becomes minimizing inverted pairs, modeling the probability that d_i outranks d_j via a smooth logistic sigmoid:

$$P(d_i \succ d_j) = \frac{1}{1 + e^{-\sigma(s_i - s_j)}}$$

The cross-entropy loss over these probabilities is smooth and differentiable.

3. **Listwise Approach and λ -Gradients (LambdaMART):** Directly optimizes the entire ranked list. **LambdaRank** introduces virtual surrogate gradients (λ_{ij}) , defined by weighting pairwise gradients by the absolute metric change $|\Delta \text{NDCG}_{ij}|$ resulting from swapping positions between d_i and d_j :

$$\lambda_{ij} = \frac{-\sigma}{1 + e^{\sigma(s_i - s_j)}} \cdot |\Delta \text{NDCG}_{ij}| \quad (1.9)$$

Combining these virtual gradients with Gradient Boosted Decision Trees produced **LambdaMART** [4], which remained the industry gold standard in web search engines for over a decade.

1.8 Evolution Toward Neural Information Retrieval

Beginning in 2018, Transformer architectures and pretrained models like BERT revolutionized Information Retrieval [5, 6]. Transformer models acquire deep semantic language representations through large-scale self-supervised pre-training, primarily **Masked Language Modeling (MLM)**: during training, a random fraction of tokens (typically 15%) is masked out with the special token [MASK], and the network learns to predict the original token via a dense classifier and softmax over the vocabulary $|V|$.

In document retrieval, neural approaches split into two fundamental architectural paradigms (Figure 1.6):

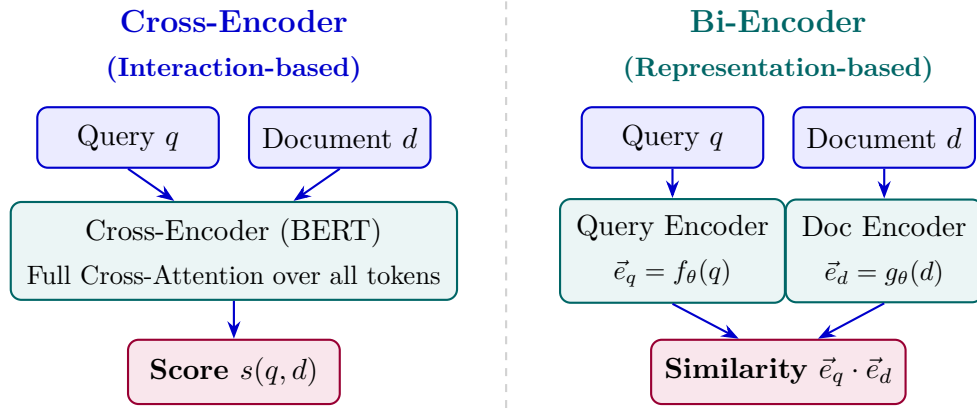


Figure 1.6: Structural comparison between Cross-Encoder (Interaction-based) and Bi-Encoder (Representation-based).

1. **Interaction-Based Models (Cross-Encoders):** The query and document are concatenated into a joint token stream:

$$\text{Input} = [\text{CLS}] \circ q \circ [\text{SEP}] \circ d$$

and fed together through all self-attention layers. This captures fine-grained cross-token interactions, delivering top-tier ranking quality.

However, its high inference cost ($\mathcal{O}(|\mathcal{D}| \times \text{Transformer})$) limits its application to second-stage re-ranking over candidate pools (e.g., top-100 results).

2. **Representation-Based Models (Bi-Encoders):** Query and document are encoded independently into vectors. Document embeddings can be indexed offline, reducing online matching to inner products or vector search.

1.8.1 Three Embedding Families

Representation models fall into three broad categories:

1. **Dense Single-Vector (e.g., DPR, Contriever):** Each document and query is mapped to a single dense vector in $\mathbb{R}^d$ ($d \approx 768$), queried via graph-based vector indices (like HNSW).
2. **Dense Multi-Vector (e.g., ColBERT [5]):** Generates token-level embeddings for query and document tokens, evaluated via **Late Interaction** using the *MaxSim* operator:

$$S(q, d) = \sum_{t_q \in q} \max_{t_d \in d} (\vec{E}_{t_q} \cdot \vec{E}_{t_d}^T) \quad (1.10)$$

Preserves token identity and achieves superior generalization, at the cost of larger index sizes.

3. **Sparse Single-Vector (e.g., SPLADE [6]):** Encodes queries and documents into high-dimensional sparse vectors over the vocabulary $|V|$ ($|V| \approx 30\,000$). Non-zero values represent learned term weights including automated term expansions.
 - **Effectiveness:** High performance across both in-domain and zero-shot out-of-domain evaluation.
 - **Efficiency:** Highly compact, directly indexable inside standard Inverted Index structures.
 - **Interpretability:** Weights align with human-readable vocabulary words, enabling transparent debugging.

Example 1.8.1 (SPLADE Semantic Expansion). Consider the natural language query:

$q = \text{"what shoes do most nba players wear"}$

A learned sparse neural retrieval model (such as SPLADE [6]) activates a selective subset of vocabulary components over $|V|$, producing a sparse vector with learned weights:

nba: 2.56, shoes: 2.32, shoe: 2.14, basketball: 1.97,
wearing: 1.55, wear: 1.52, most: 1.34, players: 1.25

This example highlights two fundamental advantages of neural sparse models:

1. **Semantic Vocabulary Expansion:** The model autonomously assigns high weight to the term "basketball" (1.97) despite its complete absence from the original query text, resolving the classical *vocabulary mismatch* problem.
2. **Morphological Harmonization:** It assigns coordinated weights to singular/plural forms (shoe / shoes) and inflected verb variations (wear / wearing), blending the advantages of linguistic stemming with deep contextual learning.

1.9 k -Approximate Nearest Neighbor Search (k -ANN) and Scalability

In high-dimensional embedding spaces, retrieval reduces to finding the top- k documents maximizing the inner product with the query vector (*Maximum Inner Product Search*, MIPS):

$$\operatorname{argmax}_{v \in \mathcal{D}}^{(k)} \vec{q}^T \cdot \vec{v} \quad (1.11)$$

On standard Web-scale collections (e.g., the MS MARCO benchmark):

- The document volume exceeds 8.8 million items ($|\mathcal{D}| \approx 8.8 \times 10^6$).
- The vocabulary vector space comprises roughly 30 000 dimensions.
- Each document activates only approximately 150 non-zero components on average.

1.9.1 Forward Index and the TusKANNy Framework

To avoid the prohibitive latency of exhaustive linear scans, modern sparse k -ANN systems combine inverted indices with a **Forward Index**. In the

Forward Index, each document is stored as a compact, contiguous sequence of (component, value) pairs:

$$d_i = \langle (c_1, v_1), (c_2, v_2), \dots, (c_m, v_m) \rangle \quad (\text{with } m \approx 150) \quad (1.12)$$

At query time ($q = \langle (c_1, v_1), \dots, (c_p, v_p) \rangle$), the engine selectively computes inner products only across overlapping non-zero components.

These techniques were highlighted in the international **NeurIPS’23 Big-ANN Challenge** and the **SISAP 2026** benchmark. In this ecosystem, **TusKANNy** (<https://tuskanny.com/>) is an open-source, high-performance Rust library co-developed by researchers from ISTI-CNR and the University of Pisa. TusKANNy implements advanced vector pruning and scanning algorithms (such as *Seismic* and *Kannolo*) capable of sustaining over 15 000 queries per second (QPS) with recall exceeding 90%, operating with sub-millisecond latencies.

Chapter 2

Evaluation of Information Retrieval Systems

2.1 Search Engine Architecture

Before diving into evaluation methodology, it is instructive to understand the high-level block diagram of a modern search engine, since each architectural component introduces measurable quality dimensions.

The main components are:

- **Web Crawler / Document Collection:** An automated agent that continuously discovers and downloads documents from the Web (or a closed corpus), feeding the indexing pipeline.
- **Parsing & Tokenization:** Documents are linguistically analyzed: HTML is stripped, text is tokenized into terms, stop-words may be removed, and stems or lemmas are computed.
- **Inverted Index:** The core data structure mapping each term to the list of documents containing it (posting list), together with positional and frequency information.
- **Query Processing & Query Expansion:** An incoming user query is tokenized with the same pipeline, possibly enriched with synonyms, related terms, or session context to improve recall.
- **Feature Extraction:** For each candidate document, a rich feature vector is assembled (BM25 score, PageRank, freshness, anchor text signals, click history, ...).

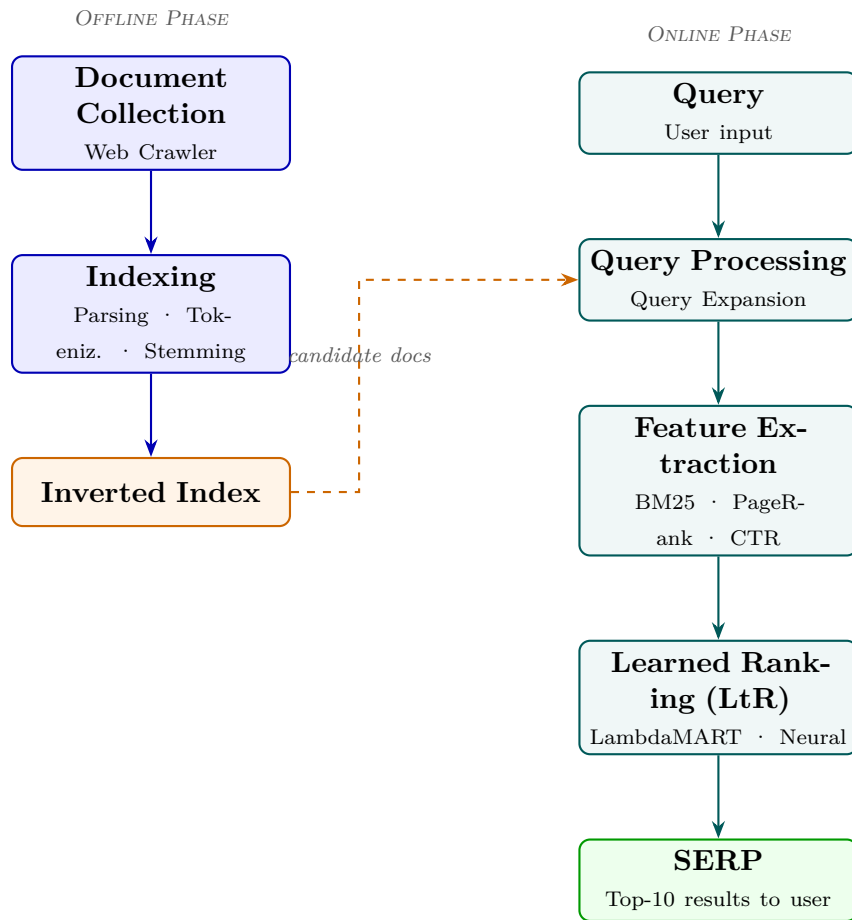


Figure 2.1: High-level block diagram of a modern search engine: offline indexing pipeline (left) and online query processing and ranking pipeline (right).

- **Learned Ranking Function (Learning to Rank):** A machine-learned model (e.g., LambdaMART, neural ranker) combines the features to produce a final relevance score used to sort the SERP.
- **SERP (Search Engine Results Page):** The final ordered list of results displayed to the user, possibly augmented with featured snippets, knowledge panels, and ads.

2.2 Evaluating Search Engine Quality

How do we evaluate such a complex, multi-component system? The answer lies in separating two complementary dimensions—**efficiency** and **effectiveness**—each requiring distinct measurement methodologies [1, 2].

- **Efficiency** concerns the *speed and resource cost* of processing: how fast can we index documents, and how quickly can we answer queries at scale?
- **Effectiveness** concerns the *quality of the answers*: are the returned documents genuinely useful to the user?

Rigorous performance evaluation is essential for designing, fine-tuning, and deploying Information Retrieval systems [1, 2]. Search engine quality is thus evaluated across these two complementary dimensions:

1. Efficiency Metrics:

- *Indexing throughput:* Number of documents processed and incorporated per hour.
- *Incremental indexing:* Ingesting new document streams (e.g., +10 000 products daily) without downtime.
- *Search latency and throughput:* Query processing response time (demanding 95th/99th percentile latencies within 10–50 ms) and CPU/memory utilization under workloads of thousands of queries per second over multi-million or multi-billion document collections.

2. Effectiveness Metrics and Auxiliary Services:

- Quality of related product recommendations, query spell-checking accuracy, and snippet generation readability.
- **Intrinsic ranking effectiveness:** Measures the engine’s core capability to satisfy users by ranking relevant documents at the top of the Search Engine Results Page (SERP).

2.3 Measuring User Happiness

In commercial Web search engines, the overarching objective is maximizing **User Happiness**. Because happiness is an internal cognitive state that cannot be directly queried, production environments rely on observable online behavioral proxies:

- **Clicks and Click-Through Rate (CTR):** The volume and proportion of clicks received by returned results.
 - *Caveat:* Misleading titles and clickbait summaries artificially inflate clicks without truly satisfying the user.
 - *Zero-Click Searches:* When is “no clicks” good news? For factual queries (e.g., weather forecasts, flight schedules, currency conversions, or dictionary definitions), answers rendered directly via *Featured Snippets* or *Knowledge Panels* satisfy the user immediately without requiring any downstream click.
- **Purchases and Economic Conversions:** In e-commerce, the percentage of search sessions resulting in transactions and total monetary spend serve as key indicators.
- **Retention and Repeat Visits:** The fraction of users returning to the search engine within a week or month.
- **Dwell Time:** The duration an attendee spends exploring a clicked page before navigating back to the SERP (*bouncing back*). Extended dwell time indicates genuine engagement, whereas near-instant returns signal dissatisfaction.

Evaluation in the Era of Generative AI and LLMs

The rise of generative conversational engines (such as ChatGPT, SearchGPT, and Perplexity) alters traditional CTR and Dwell Time dynamics: users consume direct synthesized summaries rather than navigating outgoing links. Consequently, factual grounding, hallucination minimization, and citation fidelity supplant traditional link click telemetry.

Several important debates have emerged in the research community regarding how to evaluate these new systems:

- **Number of conversational turns as a failure proxy (raised by student Dino):** In a generative dialogue system, a high number of follow-up turns might indicate that the initial response was unsatisfactory—the user is forced to re-ask or clarify. Conversely, resolving the need in a single turn could signal high effectiveness. This is an emerging behavioral proxy with no settled consensus.
- **Prompting variance issue (raised by student Gabriele):** Unlike a keyword query submitted to a classical IR engine, the phrasing of a prompt to a generative LLM can dramatically change the output quality. This makes reproducible evaluation extremely difficult: the same underlying information need can be expressed in thousands of ways, each yielding a different response.
- **Ontological difference from classic IR (raised by student Bruno):** Traditional IR retrieves *existing* documents; generative LLMs *synthesize* new text. This is a fundamentally different epistemic act—the system is no longer a retrieval engine but a generator, making relevance judgments harder to ground in objective fact.
- **Retrieval-Augmented Generation (RAG):** One prominent engineering paradigm to mitigate hallucinations is RAG, in which a generative model is provided with retrieved documents as grounding context before generating its response. RAG partially bridges the gap between classical IR and LLMs, but introduces new evaluation challenges: both the retrieval step and the generation step must be assessed independently and jointly.

2.4 The Cranfield Paradigm and Offline Test Collections

While online signals offer valuable telemetry, they are noisy, subject to position bias, and unsuitable for comparing algorithmic improvements offline prior to production deployment.

To situate the Cranfield approach historically: Cyril Cleverdon conducted his seminal experiments in the **1960s**, working with *physical* library systems—card catalogues and aeronautical engineering records at the Cranfield Institute of Technology—long before digital text retrieval was computationally feasible. His insight was to construct a controlled experimental collection with exhaustive relevance judgments, providing the first rigorous laboratory for IR comparison. This setting, humble by today’s standards, established a scientific template still followed six decades later.

To establish reproducible, controlled evaluation, IR research adheres to the **Cranfield Paradigm**, pioneered by Cyril Cleverdon in the 1960s [7].

Definition 2.4.1 (Test Collection). A standard Information Retrieval test collection is a formal triplet $\mathcal{T} = (\mathcal{D}, \mathcal{Q}, \mathcal{J})$ consisting of:

1. A benchmark document collection $\mathcal{D} = \{d_1, d_2, \dots, d_N\}$;
2. A benchmark suite of queries $\mathcal{Q} = \{q_1, q_2, \dots, q_M\}$;
3. A set of relevance judgments (*qrels*) $\mathcal{J} = \{(q, d) \mapsto r_{q,d}\}$, assigning a ground-truth relevance assessment (binary or graded) to each query-document pair.

2.4.1 Combinatorial Scale Bottlenecks

Exhaustively assessing all query-document pairs is physically infeasible for realistic corpus sizes. Consider a representative benchmark with:

$$|\mathcal{D}| = 5 \times 10^6 \text{ documents, } |\mathcal{Q}| = 50\,000 \text{ queries}$$

The full Cartesian product comprises:

$$|\mathcal{D}| \times |\mathcal{Q}| = 5 \times 10^6 \times 50\,000 = 2.5 \times 10^{11} \text{ pairs } (0.25 \text{ trillion judgments})$$

Assuming an experienced human assessor evaluates one document in 2.5 seconds:

$$\begin{aligned} \text{Total Time} &= 2.5 \times 10^{11} \times 2.5 \text{ s} = 6.25 \times 10^{11} \text{ seconds} \\ &\approx 173\,611\,111 \text{ person-hours} \end{aligned}$$

This would require tens of thousands of human working years! Consequently, benchmark datasets must restrict assessment to a sampled subset of candidate documents.

2.4.2 Origins of Test Queries

A natural question is: *where do the queries in a benchmark test suite come from?* The answer differs significantly depending on the evaluation context:

- **Web search engines — Query Logs:** Large-scale industrial benchmarks (such as those used in Microsoft TREC tracks or the AOL query log dataset) sample queries directly from *real user query logs*. This ensures that the test suite faithfully reflects the actual distribution of user information needs, including rare and long-tail queries.
- **Classic laboratory scenarios — Expert-drafted lists:** In pre-Web IR benchmarks (Cranfield, TREC ad hoc tracks), domain experts *manually composed* a set of information need statements (called *topics*). Each topic is a verbose narrative describing the underlying information need, which is then abbreviated into a short keyword query for system input.
- **What is *not* done — Random queries from documents:** One might naively suggest generating queries by sampling random terms from documents in the collection. This approach is **explicitly avoided** in IR evaluation: randomly sampled terms do not resemble real user intents, would introduce severe artificial biases, and would not generalize to the retrieval scenarios that actually matter to end users.

2.4.3 Crowdsourcing: Opportunities and Pitfalls

One cost-effective alternative delegates judging to online micro-task platforms (e.g., Amazon Mechanical Turk). However, empirical literature indicates that crowdsourced annotations suffer from:

- Substantially higher variance compared to domain experts;
- Inconsistent quality and exposure to adversarial spamming;
- A requirement for robust filtering mechanisms (majority voting, worker reputation scoring, Cohen’s κ inter-rater reliability thresholds).

A more recent and increasingly adopted alternative is **LLM-as-a-Judge**:

- **LLM-as-a-Judge:** State-of-the-art language models (such as GPT-4-class models) are prompted with structured evaluation rubrics to assign relevance grades to query-document pairs. This approach offers substantial *economic scalability*—millions of judgments can be obtained at a fraction of the cost of human crowdworkers. However, it introduces well-documented *intrinsic biases*: **length bias** (longer, more verbose responses tend to receive higher scores regardless of content quality), **self-preference bias** (a model tends to rate its own outputs more favorably), and **position bias** (the order in which candidates are presented influences the judgment). Active research is ongoing to quantify and mitigate these distortions.

2.4.4 The TREC Infrastructure

Organized by the National Institute of Standards and Technology (NIST), the **Text REtrieval Conference (TREC)** provides the foundational infrastructure for large-scale IR experimentation. TREC is organized into specialized thematic *tracks* reflecting distinct information access tasks (Web search, biomedical IR, conversational search, deep learning retrieval). The output produced by a participant system executing a retrieval task on a test collection is termed a **Run**.

Table 2.1 documents the dimensional evolution of classical benchmark collections across Information Retrieval history [1], emphasizing the massive scale increase that necessitated the creation of TREC.

Collection	Documents (N)	Queries	Size (MB)	Terms/Doc	Assessments (Q-D)
ADI	82	35	—	—	—
AIT	2 109	14	2	400.0	> 10 000
CACM	3 204	64	2	24.5	—
CISI	1 460	112	2	46.5	—
Cranfield	1 400	225	2	53.1	Exhaustive
LISA	5 872	35	3	—	—
Medline	1 033	30	1	—	—
NPL	11 429	93	3	—	—
OSHMED	348 566	106	400	250.0	16 140
Reuters	21 578	672	28	131.0	—
TREC (Typical)	740 000	200	2 000 (2 GB)	89–3 543	> 100 000

Table 2.1: Comparative overview of classical historical evaluation test corpora.

As evident from the empirical data, early corpora from the 1960s and 1970s (such as Cranfield and Medline) comprised only a few thousand records and 1–2 MB, allowing exhaustive manual judging. With the advent of realistic Web-scale corpora containing hundreds of thousands of documents

and gigabytes of text like TREC, the combinatorial evaluation space exploded, mandating selective sampling strategies.

2.4.5 The Pooling Technique

To select which documents human assessors should judge without evaluating the entire corpus, TREC standardizes the **Pooling** methodology (Figure 2.2):

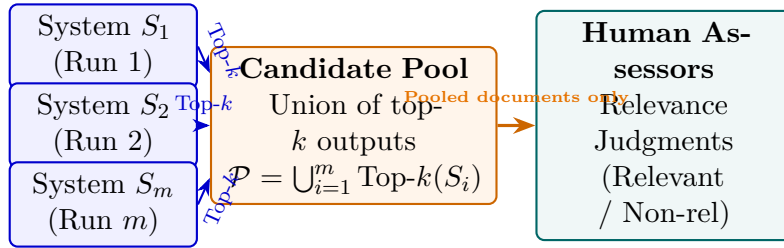


Figure 2.2: The Pooling technique for selecting candidate documents for human judgment.

1. Collect ranked result lists from m diverse retrieval systems for a given topic.
2. Extract the top- k ranked documents from each run (typically $k = 50$ or $k = 100$).
3. Take the **union** of these candidate sets, deduplicating documents to build pool $\mathcal{P}$.
4. Human judges assess relevance strictly for documents inside $\mathcal{P}$.

The Unjudged Document Assumption

All documents in the corpus not present in the pool (*unpooled*) are formally assumed to be **non-relevant** ($r = 0$). While this assumption may theoretically penalize innovative systems surfacing relevant documents overlooked by historical pool contributors, NIST experiments confirm that sufficiently diverse pools maintain stable, fair, and reliable system rankings.

2.5 Information Need vs. Query

A crucial principle in evaluation separates the underlying need from its textual expression:

- **Information Need:** The user’s actual cognitive desire. Example: “*My swimming pool bottom is becoming black and needs to be cleaned*”.
- **Query:** The keyword shorthand submitted to the system. Example: “pool cleaner”.

Property 2.5.1 (Evaluation Principle). *Document relevance must always be assessed with respect to the user’s underlying **Information Need**, never purely against the superficial keywords of the query string.*

2.6 Unranked Retrieval Measures

In unranked evaluation contexts (e.g., Boolean retrieval), the engine returns an unordered set of documents. Performance is derived from the standard contingency table (Table 2.2):

	Document Relevant	Document Non-Relevant
Document Retrieved	True Positive (TP)	False Positive (FP)
Document Not Retrieved	False Negative (FN)	True Negative (TN)

Table 2.2: Contingency matrix for binary query-document classification.

2.6.1 Precision, Recall, and the Precision-Recall Trade-off

Definition 2.6.1 (Precision (P)). The fraction of retrieved documents that are relevant:

$$P = \frac{TP}{TP + FP} = P(\text{relevant} \mid \text{retrieved}) \quad (2.1)$$

Definition 2.6.2 (Recall (R)). The fraction of relevant documents in the collection that were successfully retrieved:

$$R = \frac{TP}{TP + FN} = P(\text{retrieved} \mid \text{relevant}) \quad (2.2)$$

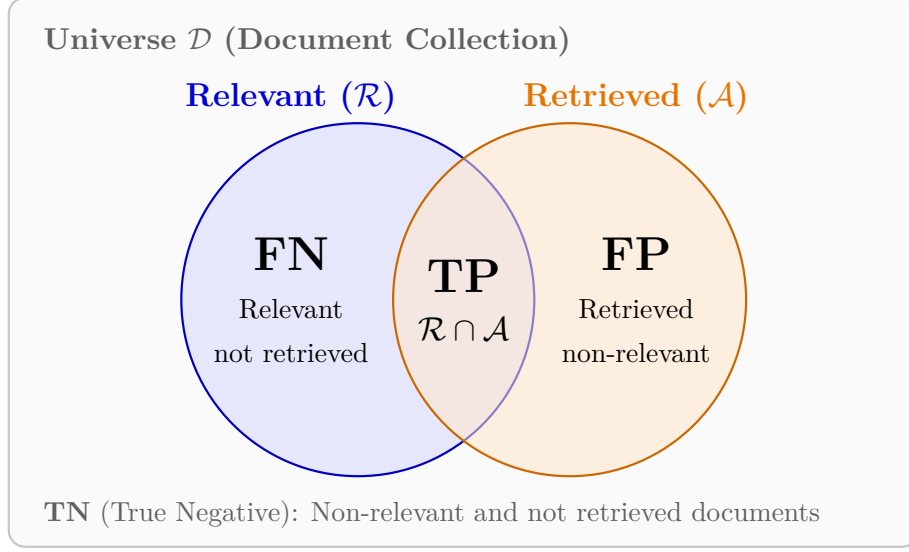


Figure 2.3: Set-theoretic Venn diagram of document retrieval and confusion regions.

An inherent **trade-off** governs Precision and Recall:

- Returning the entire document repository (Retrieved = $\mathcal{D}$) guarantees $R = 100\%$, but drops precision near zero ($P \approx 0$).
- Conversely, returning only the single most confident document can yield $P = 100\%$, at the expense of dismal recall.

2.6.2 The Harmonic Mean: F-Measure (F_1 -Score)

To balance Precision and Recall into a single scalar metric that penalizes extreme disparity, the **F-Measure** calculates their **harmonic mean**:

$$F_1 = \frac{2}{\frac{1}{P} + \frac{1}{R}} = \frac{2 \cdot P \cdot R}{P + R} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN} \quad (2.3)$$

Property 2.6.1 (Harmonic Mean Properties). *The harmonic mean exhibits key mathematical properties for evaluation:*

1. *It is always upper-bounded by both the geometric and arithmetic means:*

$$\frac{2}{\frac{1}{a} + \frac{1}{b}} \leq \sqrt{a \cdot b} \leq \frac{a + b}{2}$$

2. When the two operands diverge significantly, the harmonic mean is heavily drawn toward the **minimum** rather than the arithmetic average.

Example 2.6.1 (Edge Behavior of F-Measure). Consider a degenerate retrieval system returning all documents: $R = 1.0$ and $P = 0.0001$.

- The arithmetic mean yields an artificially inflated score:

$$\frac{1.0 + 0.0001}{2} \approx 0.50 \quad (50\%)$$

- The harmonic mean (F_1) penalizes the system:

$$F_1 = \frac{2 \cdot 1.0 \cdot 0.0001}{1.0 + 0.0001} \approx 0.0002 \quad (0.02\%)$$

This illustrates why the harmonic mean is mathematically favored over the arithmetic mean.

2.6.3 The Weighted F_β -Measure

When asymmetric priority is given to Precision versus Recall, a weighting parameter $\alpha \in [0, 1]$ is introduced:

$$F = \frac{1}{\alpha \frac{1}{P} + (1 - \alpha) \frac{1}{R}} \quad (2.4)$$

Setting $\alpha = \frac{1}{1+\beta^2}$ leads to the standard F_β formulation:

$$F_\beta = \frac{(1 + \beta^2) \cdot P \cdot R}{\beta^2 \cdot P + R} \quad (2.5)$$

- Values of $\beta < 1$ (e.g., $\beta = 0.5$) emphasize **Precision** over Recall (favored in Web search).
- Values of $\beta > 1$ (e.g., $\beta = 2.0$) emphasize **Recall** over Precision (crucial in legal discovery, patent search, and clinical meta-analyses).

2.7 Rank-Based Measures (Binary Relevance)

Modern search engines return ranked lists ordered by descending relevance scores. Furthermore, Web users rarely look beyond the first page of results ($k = 10$ or $k = 30$).

2.7.1 Precision@k ($P@k$) and Recall@k ($R@k$)

Definition 2.7.1 (Precision@k). Given a fixed rank cutoff $k \in \mathbb{N}^+$, Precision@k is the fraction of relevant documents within the top k positions:

$$P@k = \frac{1}{k} \sum_{i=1}^k \text{rel}_i \quad (2.6)$$

where $\text{rel}_i \in \{0, 1\}$ represents the binary relevance of the document at rank i .

Example 2.7.1 (Computing Precision@k). Consider a ranked list with binary relevance sequence for the top 5 positions $\langle 1, 1, 0, 0, 1 \rangle$:

- $P@1 = \frac{1}{1} = 1.00$
- $P@2 = \frac{2}{2} = 1.00$
- $P@3 = \frac{2}{3} \approx 0.66$
- $P@4 = \frac{2}{4} = 0.50$ (note that $P@4 < P@3$)
- $P@5 = \frac{3}{5} = 0.60$

The Distortion of $P@k$ When Averaged Across Queries

While $P@k$ is intuitive for estimating top-page result quality, it suffers from a critical methodological flaw when averaged across heterogeneous queries:

- If a specific query q_A possesses only $R_A = 2$ relevant documents in the entire corpus, the theoretical maximum possible $P@10(q_A)$ is capped at $\frac{2}{10} = 0.20$, even if a flawless engine places both relevant documents at ranks 1 and 2!
- Averaging raw $P@10$ across queries penalizes the retrieval engine for the intrinsic scarcity of relevant documents for specific information needs, rather than algorithmic ranking errors.

Similarly, the complementary measure *Recall@k* ($R@k$) evaluates the fraction of all relevant documents recovered within rank k :

$$\text{Recall@}k = \frac{1}{R} \sum_{i=1}^k \text{rel}_i \quad (2.7)$$

2.7.2 Average Precision (AP)

Average Precision (AP) summarizes the entire Precision-Recall curve into a single scalar score for a given query, evaluating instantaneous precision at each rank position where recall increases.

Definition 2.7.2 (Average Precision). For a ranking of N documents corresponding to a query with R total relevant documents in the collection:

$$AP = \frac{1}{R} \sum_{k=1}^N P@k \cdot \text{rel}_k \quad (2.8)$$

Property 2.7.1 (Geometric Interpretation and the Role of $1/R$). *Average Precision carries a direct analytical meaning:*

1. *Each time a relevant document is encountered at rank k , recall increases by a discrete increment $\Delta\text{Recall} = \frac{1}{R}$. AP accumulates the rectangular areas $P@k \cdot \Delta\text{Recall}$, representing the **discrete numerical approximation of the Area Under the Precision-Recall Curve** (AUC-PR).*
2. *Normalizing by R (the total number of relevant documents in the corpus) ensures that relevant documents omitted from the top- N candidate pool implicitly contribute zero precision, heavily penalizing systems with incomplete recall.*

	Search Engine Output Order →				
Rank Position (k):	$k = 1$	$k = 2$	$k = 3$	$k = 4$	$k = 5$
Document:	Rel	NonRel	Rel	NonRel	Rel
Precision@ k :	1.00	—	0.67	—	0.60

Figure 2.4: Schematic representation of Average Precision calculation over 5 documents with $R = 3$ total relevant items.

Example 2.7.2 (Basic Calculation with $R = 3$ Relevant Documents). Consider the ranking scenario in Figure 2.4 with $R = 3$ total relevant documents appearing at ranks $k_1 = 1$, $k_2 = 3$, and $k_3 = 5$:

$$P@1 = \frac{1}{1} = 1.00, \quad P@3 = \frac{2}{3} \approx 0.667, \quad P@5 = \frac{3}{5} = 0.60$$

The resulting Average Precision is:

$$AP = \frac{1}{3} \left(1.00 + \frac{2}{3} + \frac{3}{5} \right) = \frac{1}{3} (2.267) \approx 0.756 \quad (75.6\%) \quad (2.9)$$

Example 2.7.3 (Comparative Analysis Between Two Rankings). Consider a query associated with $R = 6$ relevant documents in the collection. Two competing search algorithms output the following top-10 ranked results:

- **Ranking #1:** Relevant items concentrated early in the list:

$\langle \text{Rel}, \text{NonRel}, \text{Rel}, \text{Rel}, \text{Rel}, \text{Rel}, \text{NonRel}, \text{NonRel}, \text{NonRel}, \text{Rel} \rangle$

- **Ranking #2:** Relevant items delayed and scattered:

$\langle \text{NonRel}, \text{Rel}, \text{NonRel}, \text{NonRel}, \text{Rel}, \text{Rel}, \text{Rel}, \text{NonRel}, \text{Rel}, \text{Rel} \rangle$

Table 2.3 presents the step-by-step evolution of Precision and Recall across all 10 ranks.

Rank (k)	Ranking #1			Ranking #2		
	Item	Recall	Precision	Item	Recall	Precision
1	Rel	0.17	1.00	NonRel	0.00	—
2	NonRel	0.17	—	Rel	0.17	0.50
3	Rel	0.33	0.67	NonRel	0.17	—
4	Rel	0.50	0.75	NonRel	0.17	—
5	Rel	0.67	0.80	Rel	0.33	0.40
6	Rel	0.83	0.83	Rel	0.50	0.50
7	NonRel	0.83	—	Rel	0.67	0.57
8	NonRel	0.83	—	NonRel	0.67	—
9	NonRel	0.83	—	Rel	0.83	0.56
10	Rel	1.00	0.60	Rel	1.00	0.60
Average Precision (AP)			0.78	0.52		

Table 2.3: Detailed comparative evaluation of Average Precision between Ranking #1 and Ranking #2.

The resulting AP scores evaluate to:

$$AP(\text{Ranking \#1}) = \frac{1.00 + 0.67 + 0.75 + 0.80 + 0.83 + 0.60}{6} = \frac{4.65}{6} \approx \mathbf{0.78}$$

$$AP(\text{Ranking \#2}) = \frac{0.50 + 0.40 + 0.50 + 0.57 + 0.56 + 0.60}{6} = \frac{3.13}{6} \approx \mathbf{0.52}$$

Although both systems achieve identical overall recall ($R = 100\%$ across 10 retrieved documents), Ranking #1 obtains a substantially higher score (0.78 vs 0.52) because it rewards front-loading relevant items.

2.7.3 Mean Average Precision (MAP)

To quantify overall effectiveness across a suite of $|\mathcal{Q}|$ benchmark queries, we compute the arithmetic mean of their respective AP scores:

Definition 2.7.3 (Mean Average Precision).

$$MAP = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} AP(q) \quad (2.10)$$

Property 2.7.2 (Macro-Averaging vs. Micro-Averaging). *MAP formally implements a **Macro-Averaging** scheme: each query carries identical statistical weight $1/|\mathcal{Q}|$.*

- *In contrast, Micro-Averaging (pooling all instantaneous precisions across all queries and dividing by the total sum of relevant documents) would be overwhelmingly dominated by a handful of broad queries (e.g., "movies" or "news", which often match thousands of relevant documents), drowning out performance on specific long-tail queries.*
- *Macro-Averaging ensures that every individual information need receives equal representation in the final benchmark metric.*

Example 2.7.4 (Computing MAP Over Two Queries). Consider an evaluation suite consisting of two distinct queries q_1 and q_2 :

- **Query 1 ($R_1 = 5$ total relevant documents):** The engine returns 5 relevant items at ranks $k \in \{1, 3, 6, 9, 10\}$ with precisions 1.00, 0.67, 0.50, 0.44, and 0.50:

$$AP(q_1) = \frac{1.00 + 0.67 + 0.50 + 0.44 + 0.50}{5} = \frac{3.11}{5} \approx \mathbf{0.62}$$

- **Query 2 ($R_2 = 3$ total relevant documents):** The engine returns 3 relevant items at ranks $k \in \{2, 5, 7\}$ with precisions 0.50, 0.40, and 0.43:

$$AP(q_2) = \frac{0.50 + 0.40 + 0.43}{3} = \frac{1.33}{3} \approx \mathbf{0.44}$$

The overall Mean Average Precision across both queries is:

$$MAP = \frac{AP(q_1) + AP(q_2)}{2} = \frac{0.62 + 0.44}{2} = \mathbf{0.53} \quad (2.11)$$

2.7.4 Truncated Ranking Measures: AP@k and MAP@k

In Web search and Recommender Systems, evaluating AP across an entire collection is impractical (the true total count R of relevant items is unknown) and disconnected from user browsing habits. Engines thus adopt rank-truncated formulations (**MAP@K**):

Definition 2.7.4 (AP@K and MAP@K). Given a fixed rank cutoff K (typically $K = 10$ or $K = 20$), Average Precision at K for query q_i is defined as:

$$AP@K(q_i) = \frac{1}{K_i} \sum_{k=1}^K P@k(q_i) \cdot \text{rel}(q_i, k) \quad (2.12)$$

where the normalizing denominator K_i reflects the number of relevant items within the top K :

$$K_i = \sum_{k=1}^K \text{rel}(q_i, k) \quad (\text{or } K_i = \min(R_i, K)) \quad (2.13)$$

The global **MAP@K** across query suite $\mathcal{Q}$ evaluates to:

$$MAP@K(\mathcal{Q}) = \frac{1}{|\mathcal{Q}|} \sum_{q_i \in \mathcal{Q}} AP@K(q_i) \quad (2.14)$$

This formulation resolves the fundamental flaw of raw $P@K$: by normalizing by K_i instead of a rigid K , a query with only 2 relevant documents that places both in the top 2 slots attains $AP@10 = 1.0$ rather than being artificially capped at 0.20.

2.7.5 Mean Reciprocal Rank (MRR)

In tasks such as known-item search, navigational queries (e.g., locating an institutional homepage), or factoid retrieval, the user is interested solely in finding the **first correct document**. User effort is inversely proportional to the rank of that first hit:

Definition 2.7.5 (Reciprocal Rank and MRR). Let $\text{rank}_1(q)$ denote the rank position of the first relevant document returned for query q . The **Reciprocal Rank** (RR) is:

$$RR(q) = \frac{1}{\text{rank}_1(q)} \quad (2.15)$$

The **Mean Reciprocal Rank** (MRR) across all queries is:

$$MRR = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \frac{1}{\text{rank}_1(q)} \quad (2.16)$$

If the first relevant hit appears at rank 1, the score is 1.0; if at rank 2, it drops to 0.5; if delayed to rank 10, it falls to 0.1.

2.8 Beyond Binary Relevance: Graded Relevance and NDCG

In practical Web and commercial applications, binary judgments prove inadequate. Consider the following real-world Google search query:

Query: "information retrieval unipi"

A commercial SERP returns pages exhibiting distinct qualitative levels of utility:

1. **Course DidaWiki (Instructor, CFU, syllabus, lecture notes):** The primary resource fully satisfying user search intent $\implies$ **Highly Relevant** ($r = 3$).
2. **Official UniPi Valutami Portal (Exam registration and administrative syllabus):** Authoritative institutional record but lacks study material $\implies$ **Relevant** ($r = 2$).
3. **Master's Degree PDF Brochure:** Contains only a brief mention of the course $\implies$ **Marginally Relevant** ($r = 1$).
4. **Unrelated articles or campus news:** $\implies$ **Irrelevant** ($r = 0$).

While all top three pages are pertinent, user satisfaction depends heavily on displaying the most informative page at rank 1. To capture this nuance, IR formalizes multi-grade relevance scales ($r \in \{0, 1, 2, 3\}$ or $\{0, \dots, 4\}$).

The **Discounted Cumulative Gain** (DCG) framework, introduced by Järvelin and Kekäläinen [8], rests on two assumptions:

1. Highly relevant documents are much more useful than marginally relevant ones.
2. The lower a document appears in the ranking, the less likely the user is to examine it; hence, gain must be discounted with descending rank.

2.8.1 Cumulative Gain (CG) and Discounted Cumulative Gain (DCG)

Definition 2.8.1 (Cumulative Gain at Rank p). Given a vector of graded relevance values $\mathbf{r} = [r_1, r_2, \dots, r_p]$, the Cumulative Gain is the unweighted sum:

$$CG_p = \sum_{i=1}^p r_i \quad (2.17)$$

CG_p is position-insensitive: swapping a highly relevant document at rank 1 with one at rank 10 leaves CG_p unchanged.

Definition 2.8.2 (Discounted Cumulative Gain (Standard Formulation)). Standard DCG applies a logarithmic discount based on rank i :

$$DCG_p = r_1 + \sum_{i=2}^p \frac{r_i}{\log_2(i)} \quad (2.18)$$

Behavioral Meaning of the Logarithmic Discount

The discount factor $\frac{1}{\log_2(i)}$ reflects the exponential decay of user visual attention under cascade reading models:

- At rank $i = 4$, the discount factor is $\frac{1}{\log_2(4)} = \frac{1}{2}$: perceived informative utility is cut in half relative to rank 1.
- At rank $i = 8$, the factor drops to $\frac{1}{\log_2(8)} = \frac{1}{3}$.
- At rank $i = 16$, the factor falls to $\frac{1}{\log_2(16)} = \frac{1}{4}$.

Commercial search engines (Google, Bing) frequently utilize the **exponential formulation**:

$$DCG_p^{\text{exp}} = \sum_{i=1}^p \frac{2^{r_i} - 1}{\log_2(i + 1)} \quad (2.19)$$

Summation Index Clarification

Note the index shift in the logarithm denominator:

- In Equation (2.18), r_1 is separated to prevent division by $\log_2(1) = 0$, while the summation starts at $i = 2$ with $\log_2(i)$.
- In Equation (2.19), the summation starts directly at $i = 1$ by taking $\log_2(i + 1)$ ($\log_2(1 + 1) = \log_2(2) = 1$).
- Furthermore, numerator $2^{r_i} - 1$ non-linearly rewards top-tier relevance ($r = 3 \implies 2^3 - 1 = 7$, vs. $r = 1 \implies 2^1 - 1 = 1$), severely penalizing any demotion of stellar results.

2.8.2 Normalized Discounted Cumulative Gain

Because different queries have varying numbers of relevant documents, raw DCG_p scores cannot be compared across queries. We normalize by the **Ideal DCG** ($IDCG_p$), obtained by computing the DCG of the ideal ranking sorted by relevance descending:

$$NDCG_p = \frac{DCG_p}{IDCG_p} \in [0, 1] \quad (2.20)$$

2.8.3 Complete Numerical Walkthrough

Consider the following 10-document ranking, judged on a scale of 0–3:

$$\mathbf{r} = [3, 2, 3, 0, 0, 1, 2, 2, 3, 0]$$

The ideal sorted ranking is:

$$\mathbf{r}^* = [3, 3, 3, 2, 2, 2, 1, 0, 0, 0]$$

Table 2.4 summarizes the step-by-step progression across all 10 ranks.

2.8.4 Comparing Alternative Ranking Functions

Consider 4 documents $\{d_1, d_2, d_3, d_4\}$ with graded assessments:

$$r(d_1) = 0, \quad r(d_2) = 1, \quad r(d_3) = 2, \quad r(d_4) = 2$$

i	r_i	Discount	Discounted Gain	$DCG@i$	r_i^*	Ideal Gain	$IDCG@i$	$NDCG@i$
1	3	1.000	3.00	3.00	3	3.00	3.00	1.00
2	2	$\log_2(2) = 1.000$	2.00	5.00	3	3.00	6.00	0.83
3	3	$\log_2(3) = 1.585$	1.89	6.89	3	1.89	7.89	0.87
4	0	$\log_2(4) = 2.000$	0.00	6.89	2	1.00	8.89	0.76
5	0	$\log_2(5) = 2.322$	0.00	6.89	2	0.86	9.75	0.71
6	1	$\log_2(6) = 2.585$	0.39	7.28	2	0.77	10.52	0.69
7	2	$\log_2(7) = 2.807$	0.71	7.99	1	0.36	10.88	0.73
8	2	$\log_2(8) = 3.000$	0.67	8.66	0	0.00	10.88	0.80
9	3	$\log_2(9) = 3.170$	0.95	9.61	0	0.00	10.88	0.88
10	0	$\log_2(10) = 3.322$	0.00	9.61	0	0.00	10.88	0.88

Table 2.4: Detailed trace of $DCG@i$, $IDCG@i$, and $NDCG@i$ computation for ranks 1 through 10.

- **Ground Truth (GT):** Ordering $\langle d_4, d_3, d_2, d_1 \rangle$ with relevance vector $[2, 2, 1, 0]$:

$$IDCG_4 = 2 + \frac{2}{\log_2(2)} + \frac{1}{\log_2(3)} + 0 = 2 + 2 + 0.6309 = 4.6309$$

$$NDCG_{GT} = 1.00$$

- **Ranking Function 1 (RF_1):** Ordering $\langle d_3, d_4, d_2, d_1 \rangle$ with relevance $[2, 2, 1, 0]$. Because the first two documents share identical relevance ($r = 2$), their permutation incurs no positional penalty:

$$DCG_{RF_1} = 2 + \frac{2}{\log_2(2)} + \frac{1}{\log_2(3)} = 4.6309$$

$$NDCG_{RF_1} = \frac{4.6309}{4.6309} = 1.00$$

- **Ranking Function 2 (RF_2):** Ordering $\langle d_3, d_2, d_4, d_1 \rangle$ with relevance $[2, 1, 2, 0]$. Here, d_2 ($r = 1$) erroneously outranks d_4 ($r = 2$):

$$DCG_{RF_2} = 2 + \frac{1}{\log_2(2)} + \frac{2}{\log_2(3)} + 0 = 2 + 1 + 1.2619 = 4.2619$$

$$NDCG_{RF_2} = \frac{DCG_{RF_2}}{IDCG_4} = \frac{4.2619}{4.6309} \approx 0.9203$$

This illustrates how NDCG mathematically penalizes rank inversions in direct proportion to the relevance difference.

Bibliography

- [1] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [2] Stefan Büttcher, Charles L. A. Clarke, and Gordon V. Cormack. *Information Retrieval: Implementing and Evaluating Search Engines*. MIT Press, 2010.
- [3] Alistair Moffat and Luke Stuvier. Binary interpolative coding for effective index compression. *Information Processing & Management*, 36(5):667–687, 2000.
- [4] Christopher J. C. Burges. From RankNet to LambdaRank to LambdaMART: An overview. Technical Report MSR-TR-2010-82, Microsoft Research, 2010.
- [5] Omar Khattab and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 39–48, 2020.
- [6] Thibault Formal, Benjamin Piwowarski, and Stéphane Clinchant. SPLADE: Sparse lexical and expansion model for information retrieval. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2288–2292, 2021.
- [7] Cyril W. Cleverdon. The Cranfield tests on index language devices. *Aslib Proceedings*, 19(6):173–194, 1967.
- [8] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of retrieval techniques. *ACM Transactions on Information Systems (TOIS)*, 20(4):422–446, 2002.